

Realizzazione di uno spazio di tuple in Erlang

Merlo Pietro

p.merlo@campus.uniurb.it

Riassunto

In questo progetto è stata realizzata un'implementazione di uno spazio di tuple (*tuple space*) con Erlang. Sono state definite delle funzionalità di base per permettere ad uno o più nodi di interagire con il tuple space, aggiungendo, leggendo e rimuovendo elementi presenti al suo interno. È stata implementata la possibilità di aggiungere e rimuovere nodi con accesso al tuple space e sono stati inseriti meccanismi che ne garantiscono il reistanziamento in caso di failure. Sono infine stati effettuati dei test relativi alle prestazioni di questo sistema.

1 Introduzione

In questo progetto viene affrontata la realizzazione di un sistema distribuito il cui scopo è la coordinazione tra processi, basato sul paradigma del *tuple space*. L'idea di base e il fine ultimo di questa implementazione è la comunicazione disaccoppiata tra processi, effettuata attraverso uno spazio di tuple condiviso, senza che tali processi conoscano direttamente i rispettivi destinatari e mittenti dei messaggi che condividono. Questo modello deriva dal paradigma *Linda*, che introduce un meccanismo di comunicazione basato sulla condivisione di tuple, inserite, lette e consumate dai processi stessi, attraverso uno spazio comune. Il progetto descritto in questa relazione è stato sviluppato in Erlang e propone di sfruttare le caratteristiche del linguaggio come concorrenza, passaggio di messaggi e supporto alla distribuzione per implementare una versione concreta di un tuple space distribuito. Il sistema in questione è stato progettato attraverso una distinzione dei diversi ruoli architetturali coinvolti, separandoli in diversi processi con mansioni differenti, come interfaccia di accesso, gestione dello stato e supervisione dei processi. Nel seguito di questo capitolo verranno descritti i concetti teorici utilizzati come il paradigma Linda e il concetto di tuple space, l'obiettivo del progetto e le caratteristiche principali di Erlang che risultano rilevanti per l'implementazione sviluppata.

1.1 Paradigma *Linda* e *tuple space*

Per contestualizzare l'ambito di questo progetto è opportuno richiamare il modello teorico da cui prende ispirazione. Questa implementazione, infatti, si basa sul paradigma Linda e sul concetto di spazio di tuple, nozioni strettamente collegate nell'ambito della programmazione concorrente e distribuita.

Il paradigma Linda nasce nel 1979 come modello di coordinazione per programmi concorrenti, viene sviluppato da David Gelernter presso la Yale University e si afferma nei successivi anni '80 come alternativa ai modelli tradizionali esistenti basati esclusivamente su memoria condivisa o *message passing*. L'idea centrale del paradigma Linda è la distinzione tra computazione e coordinazione: i processi mantengono la propria esecuzione in modo indipendente mentre la loro cooperazione viene affidata ad un meccanismo comune a sé stante. In questo modo, non è richiesta nessuna conoscenza reciproca né alcun tipo di sincronizzazione tra i vari processi, i quali però possono interagire in maniera disaccoppiata, sia dal punto di vista spaziale che dal punto di vista temporale. Questo approccio rende il paradigma Linda particolarmente adatto all'utilizzo in sistemi distribuiti, nei quali risulta molto importante la flessibilità della comunicazione e la riduzione delle dipendenze dirette tra i vari componenti che fanno parte del sistema.

Il meccanismo più classico con cui il paradigma Linda realizza questo tipo di coordinazione tra processi è il *tuple space*, o in italiano, spazio di tuple, rappresentato in figura [1].

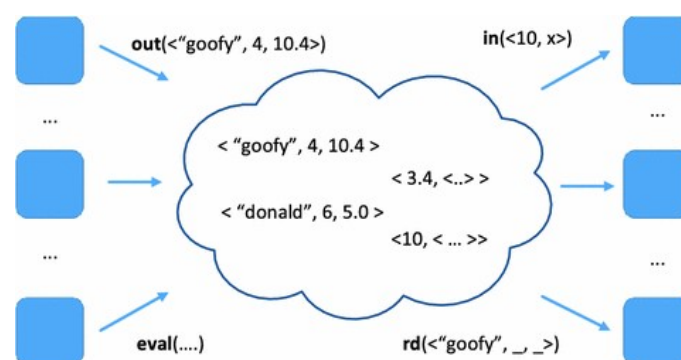


Figura 1: rappresentazione tuple space

Il tuple space è un'implementazione di una memoria condivisa che contiene tuple accessibili da più processi concorrenti. Con il termine *tuple* si intende una sequenza ordinata di campi mentre lo spazio di tuple può essere visto come una *blackboard*, o *lavagna condivisa*, sulla quale diversi processi possono depositare e

recuperare informazioni di vario genere, sottoforma, appunto, di tupla. L'accesso ai dati non avviene tramite il loro indirizzo, come in una memoria tradizionale, ma direttamente tramite il contenuto dei dati stessi: il processo richiedente specifica un *pattern* relativo al contenuto dei dati che sta cercando e il sistema si occupa di ricercare, sulla lavagna condivisa, una tupla compatibile con tale *pattern*.

Le operazioni fondamentali del modello tuple space sono *out*, che inserisce una tupla all'interno dello spazio, *in*, che legge e rimuove una tupla compatibile con un determinato *pattern*, *rd*, che legge una tupla compatibile senza rimuoverla ed *eval*, che permette di inserire nello spazio una tupla i cui valori non sono ancora noti nel momento del suo inserimento ma verranno calcolati in seguito. In particolare, le operazioni *in* e *rd* possono essere bloccanti, ovvero bloccano l'esecuzione del processo chiamante il quale rimane in attesa fino a quando non compare, all'interno del tuple space, una tupla compatibile con la richiesta effettuata. È proprio per questa ragione che il tuple space non rappresenta soltanto un mezzo che permette a processi diversi di scambiare dati ma fornisce anche un meccanismo di sincronizzazione tra processi concorrenti.

1.2 Obiettivo del progetto

Nello specifico, l'obiettivo di questo progetto è la realizzazione in Erlang di un'implementazione di un tuple space che permetta a più processi di comunicare ed eventualmente coordinare le proprie esecuzioni tramite uno spazio condiviso di tuple, accessibile da diversi nodi di una rete. Questa implementazione fornisce la possibilità di creare e distruggere un tuple space tramite le operazioni *new(Name)* e *stop(Name)* nonché di interagire con il tuple space tramite operazioni come *out*, *in* e *rd*: l'operazione *out(TS, Tuple)* inserisce la tupla *Tuple* all'interno del tuple space e restituisce un esito positivo in caso di scrittura andata a buon fine; *in(TS, Pattern)* e *rd(TS, Pattern)* sono operazioni di lettura bloccanti che sospendono l'esecuzione del processo chiamante fino a che non compare, all'interno del tuple space, una tupla compatibile con il *Pattern* richiesto. Quando una tupla compatibile viene trovata, *rd* la restituisce al processo chiamante lasciandola presente all'interno del tuple space mentre *in*, la consuma, ovvero la restituisce al processo chiamante e la rimuove dal tuple space. Per queste ultime due operazioni sono presenti le relative versioni con timeout: *in(TS, Pattern, Timeout)* e *rd(TS, Pattern, Timeout)*. Il loro comportamento è omologo alle versioni descritte precedentemente ma in questo caso viene specificato il tempo massimo di attesa *Timeout*. In caso di successo, al processo chiamante viene restituita una tupla compatibile, mentre allo scadere del tempo di timeout, se nessuna tupla compatibile viene trovata all'interno del tuple space, viene restituito il messaggio *{err, timeout}*. Rispetto alle formulazioni generiche del modello Linda, questa implementazione non include l'operazione *eval*.

Accanto alle operazioni sulle tuple, in questo progetto è prevista la presenza di funzioni dedicate alla gestione della visibilità del tuple space sui nodi della rete. Tramite le operazioni *addNode(TS, Node)* e *removeNode(TS, Node)*, il nodo *Node* viene aggiunto o rimosso dall'insieme dei nodi autorizzati ad interagire con il tuple space *TS*; infine, *nodes(TS)* permette di ottenere la lista di nodi autorizzati.

1.3 Erlang

Erlang è un linguaggio di programmazione funzionale progettato per sviluppare sistemi concorrenti e distribuiti, concepito nel 1986 da Joe Armstrong, Robert Virding e Mike Williams, viene reso pubblico come software open source nel 1998. Il codice Erlang viene eseguito all'interno di una *virtual machine* ovvero un ambiente di esecuzione dedicato che gestisce gli aspetti fondamentali del linguaggio come processi, comunicazione e concorrenza. I processi in questione si possono definire leggeri in quanto non sono processi del sistema operativo nel senso tradizionale del termine ma bensì delle unità di esecuzione interne alla virtual machine, le quali possiedono un basso costo di creazione e terminazione, nonché un consumo di memoria ridotto. A proposito di memoria, i processi Erlang, non condividono memoria direttamente ma possono comunicare tra di loro attraverso l'invio e la ricezione di messaggi, semplificando la gestione di processi concorrenti. Un ulteriore elemento rilevante di questo linguaggio è il cosiddetto *pattern matching*, tecnica che permette di riconoscere, utilizzare e manipolare dati strutturati sulla base della loro forma, particolarmente adatto all'utilizzo con tuple, liste, messaggi e funzioni.

Queste caratteristiche si adattano particolarmente bene allo sviluppo di un tuple space distribuito: il sistema deve infatti garantire accessi concorrenti da parte di più processi, operazioni bloccanti come *in* e *rd*, ricerca tramite *Pattern*, comunicazione tra componenti e la possibilità di estendere il servizio a diversi nodi di una rete, tutti aspetti per i quali Erlang offre un modello naturale e di semplice implementazione.

2 Implementazione

L'implementazione del tuple space distribuito, oggetto di questo progetto, è suddivisa in tre moduli principali e ognuno di essi svolge un compito specifico all'interno del sistema: il modulo *tuple_space.erl* rappresenta l'interfaccia pubblica tramite la quale i processi possono interagire con lo spazio di tuple, *ts_owner.erl* gestisce lo stato effettivo del tuple space mentre *ts_supervisor.erl* si occupa dell'avviamento del sistema, di inoltrare i messaggi ai processi in questione e di mantenere il controllo sul funzionamento del processo contenente lo stato effettivo

dello spazio di tuple. Il comportamento complessivo di questo sistema si basa quindi sulla cooperazione e il coordinamento di questi tre moduli separati, i quali operano come processi distinti ma la cui esecuzione è strettamente coordinata. Nel prosieguo di questa relazione faremo riferimento a questi tre moduli con la nomenclatura “*tuple_space*”, “*supervisor*” e “*owner*”.

Per creare un nuovo spazio di tuple, il client invoca l'operazione *new(Name)* del modulo *tuple_space*. Questa richiesta provoca l'avvio del *supervisor* associato al tuple space appena inizializzato, il quale viene registrato con il nome indicato e durante la propria fase di inizializzazione, crea a sua volta il corrispondente processo *owner*. È proprio il modulo *owner* che apre il file *dets* associato al tuple space, recupera eventuali tuple e lista di nodi autorizzati già presenti all'interno della tabella ed entra nel ciclo principale di ricezione e gestione delle richieste. A questo punto, il sistema dispone di un processo di supervisione attivo, di un processo *owner* incaricato di mantenere e gestire lo stato dello spazio di tuple e di un'interfaccia pubblica tramite la quale il client può interagire con il tuple space.

Per quanto riguarda la fase di utilizzo, il client continua ad interagire esclusivamente con il modulo *tuple_space*, il quale, dopo aver ricevuto una richiesta da parte del client, costruisce il messaggio corrispondente all'operazione richiesta e, dopo aver individuato il processo destinatario associato al tuple space, recapita tale messaggio al processo supervisor corrispondente. Il processo *supervisor* inoltra il messaggio al processo *owner* il quale esegue concretamente le operazioni su tuple e nodi: inserisce nuove tuple, cerca tuple corrispondenti in caso di operazioni *in* e *rd*, gestisce eventuali richieste bloccate in attesa, aggiorna l'elenco dei nodi autorizzati e salva le modifiche sullo storage *dets* persistente. Dopo aver completato l'operazione, il modulo *owner* ne invia l'esito, sempre tramite messaggio, direttamente all'interfaccia *tuple_space* che ha effettuato la richiesta, la quale poi lo comunica al client richiedente.

Casi particolari di questo flusso operativo sono le operazioni di arresto del servizio e la gestione di eventuali guasti del sistema. Quando il client invoca l'operazione *stop(TS)*, la richiesta viene ancora una volta inoltrata al supervisor il quale invia un messaggio di arresto al processo *owner*. In questo caso, il processo *owner* si limita a chiudere la tabella *dets* corrispondente, mentre il processo *supervisor* rimuove le registrazioni del tuple space e risponde al chiamante. Se il processo *owner*, invece, termina in modo inatteso durante la sua normale esecuzione, il processo supervisor intercetta l'evento e avvia un nuovo processo *owner*, mantenendo attivo il servizio lato client.

Nei paragrafi successivi verranno descritti nel dettaglio i singoli moduli, analizzandone il codice nello specifico e descrivendone il comportamento all'interno del sistema tramite alcuni frammenti di codice.

2.1 tuple_space.erl

Il modulo *tuple_space* rappresenta l'interfaccia pubblica di questo sistema e fornisce al client la possibilità di interagire con il tuple space. Questo modulo, come accennato in precedenza, non gestisce lo stato interno dello spazio di tuple ma il suo scopo è quello di ricevere le richieste del client, individuare il destinatario corretto e inoltrare i messaggi necessari per l'esecuzione dell'operazione richiesta.

Le funzioni esportate da questo modulo, riportate in figura [2], permettono al client di interagire con lo spazio di tuple e il loro funzionamento rispetta le descrizioni citate in precedenza.

```
-module(tuple_space).  
  
-export([  
    new/1, stop/1,  
    out/2, in/2, in/3, rd/2, rd/3,  
    addNode/2, removeNode/2, nodes/1  
]).
```

Figura 2: dichiarazione modulo *tuple_space* e funzioni esportate

Come descritto precedentemente, questo modulo non gestisce direttamente lo stato interno del tuple space. La funzione *new(Name)*, come mostrato in figura [3], delega l'avvio del servizio al modulo *supervisor*, il quale, come vedremo in seguito, avvierà un processo *owner* e registrerà globalmente il nome ricevuto dal modulo *tuple_space*.

```
new(Name) ->  
    ts_supervisor:start_link(Name).
```

Figura 3: funzione *new* di *tuple_space*

A questo punto diventa necessaria la funzione *dest(TS)*, riportata in figura [4], non esportata dal modulo poiché il suo utilizzo resterà sempre interno a questo processo. Questa funzione ha il compito di individuare il *PID* del processo *supervisor* associato al tuple space richiesto, ricercandolo prima localmente, poi se necessario, globalmente. Grazie a questa funzione, il modulo *tuple_space* è in grado di individuare il destinatario corretto a cui inoltrare le richieste successive. Qualora il tuple space richiesto non venga trovato, la funzione *dest(TS)* genererà un'eccezione di tipo *exit*, interrompendo la chiamata in corso.

```

dest(TS) ->
  case whereis(TS) of
    undefined ->
      case global:whereis_name({ts, TS}) of
        undefined ->
          exit({unknown_tuple_space, TS});
        Pid ->
          Pid
      end;
    Pid ->
    Pid
  end.

```

Figura 4: funzione interna *dest(TS)*

Le altre funzioni presenti in questo modulo seguono uno schema implementativo piuttosto simile. In generale, viene creato un riferimento univoco per identificare la richiesta da parte del client tramite la funzione *make_ref()*, a seguire, tramite la funzione *dest(TS)* descritta in precedenza, viene individuato il destinatario della richiesta al quale viene inviato un messaggio contenente l'operazione richiesta e infine si attende la risposta associata al riferimento corrispondente. In figura [5] è riportata la funzione *in(TS, Pattern, Timeout)* come esempio significativo: questa funzione, nello specifico, attende la risposta da parte del processo *supervisor* per il tempo specificato da *Timeout*. Qualora non dovesse ricevere nessuna risposta in questo lasso di tempo, il modulo invia un messaggio di cancellazione della richiesta, la quale altrimenti rimarrebbe nella lista delle richieste in attesa, come vedremo in seguito. Infine, restituisce un messaggio di errore al chiamante e termina l'esecuzione della chiamata.

```

in(TS, Pattern, Timeout) ->
  Ref = make_ref(),
  Dest = dest(TS),
  Dest ! {self(), {in, Ref, Pattern}},
  receive
  | {Ref, Reply} -> Reply
  after Timeout ->
    Dest ! {cancel, self(), Ref},
    {err, timeout}
  end.

```

Figura 5: funzione *in* con timeout

2.2 ts_supervisor.erl

Il modulo *ts_supervisor* ha il compito di avviare e supervisionare il processo *owner* fungendo da intermediario tra l'interfaccia con cui comunica il client e il processo che gestisce concretamente lo stato del tuple space. Le funzioni esportate, riportate in figura [6] sono *start_link/1*, *init/1* e *loop/2* il cui compito è rispettivamente l'avvio del supervisor, la sua inizializzazione e la gestione dei messaggi che riceve sia dal processo *tuple_space* che dal processo *owner* che monitora.

```
-module(ts_supervisor).  
  
-export([start_link/1, init/1, loop/2]).
```

Figura 6: dichiarazione modulo *ts_supervisor* e funzioni esportate

La funzione *start_link/1*, in figura [7], è la funzione di avvio del modulo *supervisor*, invocata dal modulo *tuple_space*. Inizialmente controlla se esiste già un processo registrato con il nome che le è stato passato tramite la funzione *global:whereis_name/1* e, in caso di esito positivo, lo comunica al processo chiamante. Se il processo non esiste, lo crea con *spawn_link/3* e lo registra sia localmente che globalmente, in modo da renderlo rintracciabile.

```
start_link(Name) ->  
  case global:whereis_name({ts, Name}) of  
    undefined ->  
      Pid = spawn_link(?MODULE, init, [Name]),  
      register(Name, Pid),  
      case global:register_name({ts, Name}, Pid) of  
        yes -> ok;  
        no -> {err, global_registration_failed}  
      end;  
    _ ->  
      {err, already_exists}  
  end.
```

Figura 7: funzione di avvio *start_link*

All'interno della funzione appena presentata, viene effettuato uno *spawn_link* nel quale viene inserita, come funzione di ingresso, la funzione *init/1*, mostrata in figura [8]. Essa viene quindi eseguita all'avvio del nuovo processo *supervisor*. Al

suo interno, imposta *trap_exit* a *true* in modo da poter intercettare la terminazione del processo *owner* come un normale messaggio e successivamente avvia tale processo tramite la funzione *ts_owner:start*. Infine, il *supervisor* entra nel suo ciclo principale *loop/2* in cui rimane in attesa dei messaggi da gestire provenienti sia da *tuple_space* che da *owner*.

```
init(Name) ->
    process_flag(trap_exit, true),
    {ok, OwnerPid} = ts_owner:start(Name),
    loop(Name, OwnerPid).
```

Figura 8: funzione di inizializzazione *init*

Il vero comportamento del processo *supervisor* è contenuto nella funzione *loop/2*, riportata in figura [9]: al suo interno il processo rimane in attesa di ricevere messaggi. Se riceve il messaggio di *stop* da parte del processo *tuple_space*, invia a sua volta un messaggio di *stop* al processo *owner*, rimuove la registrazione del suo nome e infine, non invocando nuovamente la funzione *loop*, il processo *supervisor* termina a sua volta. Quando il processo *owner* corrispondente termina la sua attività, *supervisor* riceve un segnale di *'EXIT'*: in caso di terminazione controllata, lo ignora, mentre in caso di terminazione anomala, avvia un nuovo processo *owner* con il medesimo nome, mantenendo il sistema attivo lato client. In caso di messaggi di qualsiasi altro tipo, come *in/out/rd* o *addNode/removeNode*, essi vengono inoltrati al processo *owner* che avrà il compito di gestirli.

```
loop(Name, OwnerPid) ->
    receive
        {'EXIT', OwnerPid, normal} -> ok;
        {'EXIT', OwnerPid, shutdown} -> ok;
        {'EXIT', OwnerPid, _Reason} ->
            {ok, NewOwnerPid} = ts_owner:start(Name),
            loop(Name, NewOwnerPid);
        {stop, From} ->
            OwnerPid ! stop,
            global:unregister_name({ts, Name}),
            unregister(Name),
            From ! {reply, ok};
    Msg ->
        OwnerPid ! Msg,
        loop(Name, OwnerPid)
end.
```

Figura 9: funzione *loop* del modulo *supervisor*

2.3 ts_owner.erl

Il modulo *ts_owner* rappresenta il componente di questo sistema che gestisce concretamente lo stato del tuple space, ricevendo messaggi dal processo di coordinamento *supervisor* ed eseguendo le operazioni richieste sullo spazio di tuple. Questo processo mantiene in memoria le tuple presenti all'interno del tuple space, la lista dei nodi autorizzati e l'insieme delle richieste rimaste in attesa.

Le funzioni esportate da questo modulo, riportate in figura [10], sono *start/1*, *init/1* e *loop/5*, tuttavia solamente *start* viene invocata dall'esterno, mentre le altre due sono utilizzate internamente durante l'avvio e l'esecuzione del processo.

```
-module(ts_owner).  
  
-export([ start/1, init/1, loop/5 ]).
```

Figura 10: dichiarazione modulo *ts_owner* e funzioni esportate

La funzione *start*, in figura [11], viene invocata dal processo *supervisor* in fase di avviamento del processo *owner*: essa si limita a creare il nuovo processo tramite *spawn_link/3* indicando *init* come funzione da eseguire e ne restituisce il *PID* al processo chiamante. La funzione *init*, sempre riportata in figura [11] contiene invece la logica di inizializzazione del processo: inizialmente apre la tabella *dets* associata al tuple space e ne recupera, tramite delle funzioni di supporto, le eventuali liste di tuple e nodi autorizzati già presenti al suo interno. Controlla se il nodo corrente è già presente nella lista dei nodi e, se necessario, lo inserisce al suo interno. Dopodiché avvia effettivamente la logica del processo invocando la funzione *loop/5*, che gestisce ricezione ed elaborazione dei messaggi. I parametri passati a *loop* rappresentano rispettivamente il nome del tuple space, la tabella *dets* di riferimento, due liste contenenti tuple correnti e nodi autorizzati e una lista vuota che verrà in seguito popolata dalla funzione stessa con le richieste in attesa.

```
start(Name) ->
  Pid = spawn_link(?MODULE, init, [Name]),
  {ok, Pid}.

init(Name) ->
  FileName = atom_to_list(Name) ++ ".dets",
  {ok, Tab} = dets:open_file(Name, [ ...
  ]),
  Tuples = load_tuples(Tab),
  Nodes0 = load_nodes(Tab),
  Nodes = case lists:member(node(), Nodes0) of
    true -> Nodes0;
    false -> [node() | Nodes0]
  end,
  ts_owner:loop(Name, Tab, Tuples, Nodes, []).
```

Figura 11: avviamento del processo, funzioni *start/1* e *init/1*

La funzione *loop*, una volta invocata, rimane in attesa dei messaggi provenienti dal supervisor e ne gestisce l'elaborazione in base al tipo di richiesta ricevuta. Il primo caso rilevante è il messaggio relativo all'operazione *in*, mostrata in figura [12]: quando il processo riceve un messaggio di questo tipo, innanzitutto verifica che il nodo del processo chiamante sia autorizzato tramite la funzione di supporto *allowed*. Successivamente, attraverso un'altra funzione di supporto *take_first_match*, ricerca la prima tupla compatibile con il *Pattern* specificato nel messaggio: se una tupla compatibile viene trovata, essa viene restituita al chiamante, rimossa dal tuple space e il nuovo stato corrente viene salvato sulla tabella *dets*. Qualora non venga trovata nessuna corrispondenza, la richiesta non fallisce ma viene inserita nella lista *Waiting*, così da poter essere soddisfatta appena verrà aggiunta una tupla compatibile. In entrambi i casi, dopo aver gestito la richiesta contenuta nel messaggio, il processo aggiorna il proprio stato interno e invoca nuovamente *loop* con i parametri aggiornati.

```

{From, {in, Ref, Pattern}} ->
  case allowed(From, Nodes) of
    true ->
      case take_first_match(Pattern, Tuples) of
        {ok, Tuple, Rest} ->
          save_tuples(Tab, Rest),
          From ! {Ref, {ok, Tuple}},
          ts_owner:loop(Name, Tab, Rest, Nodes, Waiting);
        not_found ->
          NewWaiting = Waiting ++ [{From, Ref, in, Pattern}],
          ts_owner:loop(Name, Tab, Tuples, Nodes, NewWaiting)
      end;
    false ->
      From ! {Ref, {err, unauthorized}},
      ts_owner:loop(Name, Tab, Tuples, Nodes, Waiting)
  end;
end;

```

Figura 12: gestione del messaggio di tipo *in*

Un ulteriore caso particolarmente rilevante è la ricezione di un messaggio relativo all'operazione *out*, riportato in figura [13]. In questo caso il processo non si limita ad aggiungere la nuova tupla allo spazio, ma si occupa anche di verificare se essa possa soddisfare e sbloccare una o più richieste tra quelle rimaste in attesa. Anche in questo caso viene effettuato un controllo sull'autorizzazione del nodo chiamante. In caso di esito positivo, viene invocata la funzione *handle_out*, mostrata in figura [14], che gestisce l'inserimento della nuova tupla: in particolare, questa funzione dapprima verifica se la tupla da inserire possa soddisfare una richiesta *in* in attesa e in quel caso la tupla viene inviata direttamente al processo che aveva inviato tale richiesta, senza essere aggiunta allo spazio di tuple. Se invece nessuna *in* sospesa può essere soddisfatta, viene effettuato un controllo sulle richieste *rd* bloccate e compatibili con tale tupla; vengono soddisfatte tutte le *rd* compatibili e solo a questo punto la tupla viene aggiunta allo spazio e il nuovo stato salvato sulla tabella *dets*. Successivamente la funzione invoca *loop*, passando come parametri le liste aggiornate.

```

{From, {out, Ref, Tuple}} ->
  case allowed(From, Nodes) of
    true ->
      {NewTuples, NewWaiting} = handle_out(Tuple, Tuples, Waiting),
      save_tuples(Tab, NewTuples),
      From ! {Ref, {ok, Tuple}},
      ts_owner:loop(Name, Tab, NewTuples, Nodes, NewWaiting);
    false ->
      From ! {Ref, {err, unauthorized}},
      ts_owner:loop(Name, Tab, Tuples, Nodes, Waiting)
  end;
end;

```

Figura 13: gestione del messaggio di tipo *out*

```

handle_out(Tuple, Tuples, Waiting) ->
  case serve_one_in(Tuple, Waiting) of
    {served, Waiting1} ->
      Waiting2 = serve_all_rd(Tuple, Waiting1),
      {Tuples, Waiting2};
    not_served ->
      Waiting2 = serve_all_rd(Tuple, Waiting),
      {Tuples ++ [Tuple], Waiting2}
  end.

```

Figura 14: funzione di supporto *handle_out*

Accanto alle funzioni principali descritte in dettaglio finora, il modulo *ts_owner* contiene anche diverse funzioni di supporto le quali realizzano i dettagli operativi necessari per il funzionamento logico del tuple space. Ad esempio, *take_first_match* e *read_first_match* effettuano la ricerca, con e senza rimozione, di tuple compatibili con un determinato pattern specificato tra i loro parametri. Le funzioni *match* e *match_list* implementano la logica di confronto tra un pattern e una tuple. *Serve_one_in* e *serve_all_rd* effettuano la gestione delle richieste bloccate in attesa, mentre *load_tuples*, *load_nodes*, *save_tuples* e *save_nodes* si occupano della persistenza locale dello stato sulla tabella *dets*. È infine stata implementata la funzione *allowed*, menzionata precedentemente in questo capitolo, la quale verifica che un determinato nodo appartenga alla lista dei nodi autorizzati ad interagire con il tuple space.

2.4 Scelte implementative e motivazioni progettuali

In questo capitolo verranno descritte brevemente alcune decisioni implementative prese durante lo sviluppo di questo sistema e, qualora fosse necessario, verranno definite anche le motivazioni principali che hanno portato a tali decisioni.

La prima scelta progettuale fondamentale è stata l'utilizzo di Erlang e in particolare l'adozione del suo modello basato su processi leggeri e sulla comunicazione tra di essi tramite messaggi. Il sistema, come descritto in precedenza, è stato infatti suddiviso in tre processi distinti, separando l'interfaccia pubblica, la supervisione e la gestione dello stato. Questa divisione consente di rispecchiare naturalmente il paradigma concorrente e distribuito del problema affrontato evitando l'utilizzo di memoria condivisa e affidando il coordinamento in modo esclusivo allo scambio di messaggi tra i processi.

Per quanto riguarda la gestione dello stato, è stato scelto di utilizzare una tabella *dets*, per memorizzare efficientemente i dati direttamente sul disco fisso e fornire, per quanto possibile, persistenza locale. In particolare, il processo *owner* salva sulla stessa tabella, il cui nome è il medesimo del tuple space, sia la lista delle tuple presenti all'interno dello spazio, sia la lista dei nodi autorizzati, in modo da poterli recuperare in caso di terminazione controllata e successiva creazione di uno spazio di tuple con il medesimo nome. Il processo *owner*, dopo la gestione di ogni richiesta ricevuta dal client, aggiorna e salva su disco una versione aggiornata della tabella *dets* quindi anche in caso di terminazione anomala del sistema, è possibile recuperare l'ultima versione sincronizzata dal processo.

Un ulteriore dettaglio, riguardante la persistenza dello stato, è stata la decisione di non salvare localmente l'intero stato del sistema: mentre la lista di tuple e la lista di nodi autorizzati vengono salvati su tabelle *dets*, la lista delle richieste bloccate in attesa viene mantenuta solamente in memoria all'interno del processo *owner*. In caso di terminazione del processo *owner* e suo successivo riavvio, tali richieste non vengono quindi recuperate dalle tabelle su disco.

A proposito di gestione delle operazioni bloccanti sospese, essa è stata implementata tramite una singola coda chiamata *Waiting* all'interno della quale vengono inserite sia richieste di tipo *in* che richieste di tipo *rd*. Ogni elemento di tale coda è una tupla in forma $\{From, Ref, in, Pattern\}$ e contiene il processo chiamante, il riferimento della richiesta, la tipologia di operazione e il pattern associato ad essa. La motivazione di questa scelta è la semplificazione dell'organizzazione interna dello stato, utilizzando una sola coda *Waiting* e delegando la logica della gestione delle richieste sospese alle funzioni di supporto.

Nel caso di un'operazione *out*, per esempio, è stato deciso di dare priorità alle richieste *in* in attesa e solo successivamente alle richieste *rd*, creando una gerarchia tra richieste sospese in base alla loro tipologia e in secondo luogo al loro momento di ricezione.

Per le operazioni *in* e *rd* è stato infatti deciso di restituire la prima tupla compatibile trovata all'interno della lista di tuple presenti nello spazio, la quale viene aggiornata aggiungendo le nuove tuple in coda. In questo modo si evita di introdurre criteri di selezione complessi come ordinamenti o priorità particolari restituendo al chiamante la prima tupla compatibile con la richiesta. Nel caso in cui più tuple siano compatibili, vista l'aggiunta in coda delle tuple nella lista dello spazio, verrà restituita al chiamante la tupla più longeva.

Un'ulteriore importante decisione presa durante l'implementazione di questo sistema riguarda la gestione delle richieste di tipo *in* o *rd* con *timeout*. In questi casi, è il modulo *tuple_space* a gestire i tempi di attesa della risposta: la richiesta viene inoltrata al processo *supervisor*, il quale la inoltra al processo *owner* che cerca, se possibile, di soddisfarla. Se non viene trovata nessuna tupla compatibile, la richiesta viene aggiunta alla coda *Waiting*. È il processo *tuple_space* che, dopo aver atteso il tempo specificato da *timeout*, se non riceve alcuna risposta, invia al supervisor un messaggio di tipo $\{cancel, self(), Ref\}$ il quale, una volta raggiunto l'*owner* provoca la rimozione della tupla dalla lista di richieste sospese. In questo modo, la logica interna del processo *owner* rimane semplice, limitandosi a reagire ai messaggi ricevuti, senza appesantirne gli oneri.

Per aumentare la robustezza e la stabilità del processo *owner*, inoltre, è stato deciso di ignorare completamente tutti i messaggi ricevuti che non riesce a riconoscere onde evitare terminazioni anomale a causa di messaggi imprevisti o malformati.

Durante la fase di utilizzo di questo tuple space, è contemplata la possibilità che dallo stesso processo chiamante, il sistema riceva più richieste concorrenti. Per distinguerle è stato implementato l'utilizzo di un riferimento univoco *Ref* creato tramite la funzione nativa del linguaggio Erlang *make_ref()*. Tale riferimento, unico appunto per definizione, viene incluso sia nel flusso di messaggi dall'interfaccia all'*owner*, sia nel flusso contrario, permettendo al sistema di associare ogni risposta alla richiesta corrispondente. Questa implementazione è particolarmente adatta nei sistemi concorrenti e asincroni, nei quali possono risultare attive molteplici operazioni contemporaneamente senza che sia presente un ordine o un tempo di completamento determinato a priori.

Infine, per rendere il tuple space rintracciabile, sia localmente che in un contesto distribuito, è stato deciso di registrarne il nome nel *supervisor* mediante registrazione sia locale che globale. In questo modo, un tuple space creato su un nodo risulterà visibile a tutti i nodi a cui è connesso, i quali, se autorizzati, potranno a loro volta interagirci.

3 Casi d'uso

In questo capitolo verranno presentati alcuni casi d'uso del tuple space sviluppato con l'obiettivo di mostrare come il sistema reagisce alle diverse richieste dei client.

Nel primo esempio, riportato in figura [15], è mostrato il comportamento del sistema quando riceve una serie di comandi da parte di un client, il quale crea il tuple space *ts1* e inserisce le tuple *{test, 100}* e *{test, 200}* al suo interno. Successivamente effettua un'operazione *rd* con pattern *{test, '_'}*: ci si aspetta che il sistema risponda con la prima tupla inserita. Dopodiché il client effettua due operazioni *in* con lo stesso pattern per leggere e rimuovere le due tuple inserite precedentemente e infine effettua un'ulteriore *rd* con *timeout* di cinque secondi per mostrare l'effettiva rimozione di tali tuple.

```
1> tuple_space:new(ts1).          5> tuple_space:in(ts1, {test, '_'}) .
ok                                {ok,{test,100}}
2> tuple_space:out(ts1, {test, 100}). 6> tuple_space:in(ts1, {test, '_'}) .
{ok,{test,100}}                  {ok,{test,200}}
3> tuple_space:out(ts1, {test, 200}). 7> tuple_space:rd(ts1, {test, '_'}, 5000).
{ok,{test,200}}                  {err,timeout}
4> tuple_space:rd(ts1, {test, '_'}) .
```

Figura 15: esempio di utilizzo standard

Nel seguente esempio, riportato in figura [16], verrà mostrato il comportamento del sistema in ambiente distribuito. In questo esempio sono presenti due nodi chiamati *n1* e *n2*: il nodo *n1* crea un tuple space chiamato *ts2* e invia un *ping* al nodo *n2* scambiando con esso le liste di nodi noti. Successivamente, il nodo *n2* prova ad interagire con tale spazio di tuple ma non essendo autorizzato riceve un messaggio di errore. Ora, il nodo *n1*, inserisce il nodo *n2* nella lista di nodi autorizzati, effettua una *rd* bloccante con pattern *{test, '_'}* e rimane in attesa di risposta. A questo punto, il nodo *n2*, autorizzato ad interagire con il tuple space *ts2*, inserirà una tupla compatibile con la richiesta del nodo *n1* il quale riceverà la risposta che attende e sbloccherà la propria esecuzione.


```

(n1@PC-di-EasyMerloSniperX)1> tuple_space:new(ts2).
ok
n1 (n1@PC-di-EasyMerloSniperX)2> net_adm:ping('n2@PC-di-EasyMerloSniperX').
pong

n2 (n2@PC-di-EasyMerloSniperX)1> tuple_space:out(ts2, {test, 300}).
{err,unauthorized}

(n1@PC-di-EasyMerloSniperX)3> tuple_space:addNode(ts2, 'n2@PC-di-EasyMerloSniperX').
ok
n1 (n1@PC-di-EasyMerloSniperX)4> tuple_space:nodes(ts2).
['n1@PC-di-EasyMerloSniperX','n2@PC-di-EasyMerloSniperX']
(n1@PC-di-EasyMerloSniperX)5> tuple_space:rd(ts2, {test, '_'})
█

n2 (n2@PC-di-EasyMerloSniperX)2> tuple_space:out(ts2, {test, 300}).
{ok,{test,300}}

n1 (n1@PC-di-EasyMerloSniperX)5> tuple_space:rd(ts2, {test, '_'})
{ok,{test,300}}

```

Figura 16: esempio di utilizzo concorrente

Nel seguente esempio invece, viene mostrata la capacità di persistenza, seppur a livello locale, del sistema sviluppato. Nell'esempio riportato in figura [17] il client crea un nuovo tuple space chiamato *ts3*, aggiunge al suo interno la tupla $\{test, 400\}$ e ne verifica la presenza con un'operazione *rd*. Successivamente lo stesso client chiude il tuple space *ts3* tramite l'operazione *stop(ts3)*. Questa operazione, come spiegato nei capitoli precedenti, causa una terminazione controllata del processo *owner* e susseguentemente una terminazione controllata del processo *supervisor*, associati al tuple space in questione. A questo punto, il client prova ad interagire con il tuple space *ts3* ma in risposta riceve l'eccezione $\{unknown_tuple_space, ts3\}$. Il tuple space chiamato *ts3* viene poi riattivato con il comando *new(ts3)* e la successiva operazione *rd* con pattern $\{test, '_'\}$ restituisce la tupla inserita inizialmente. Questo dimostra che il sistema, una volta chiuso un tuple space, ne salva lo stato attuale localmente su disco con tabelle *dets*, fatta eccezione per la lista delle richieste in attesa. Successivamente, qualora venga aperto un tuple space con il medesimo nome, i dati relativi ad esso vengono recuperati dalla stessa tabella.

```

1> tuple_space:new(ts3).
ok
2> tuple_space:out(ts3, {test, 400}).
{ok,{test,400}}
3> tuple_space:rd(ts3, {test, '_'})
{ok,{test,400}}
4> tuple_space:stop(ts3).
ok

5> tuple_space:rd(ts3, {test, '_'})
** exception exit: {unknown_tuple_space,ts3}
6> tuple_space:new(ts3).
ok
7> tuple_space:rd(ts3, {test, '_'})
{ok,{test,400}}

```

Figura 17: esempio di terminazione controllata

4 Analisi delle prestazioni

Per valutare le prestazioni di questo sistema è stato utilizzato un ulteriore modulo Erlang chiamato *bench.erl* il quale esegue una serie di misurazioni temporali sulle operazioni fondamentali del tuple space. Le operazioni prese in considerazione sono le operazioni sulle tuple di *out*, *rd* e *in* e più in generale la fase di recovery.

Nel caso dell'operazione *out* il test prevede di misurare il tempo trascorso tra la chiamata *tuple_space:out* e la ricezione della risposta. Dopo aver inserito la tupla all'interno dello spazio, essa viene rimossa e il test viene ripetuto.

Nel caso delle operazioni *rd* e *in* il comportamento del modulo *bench* è piuttosto simile: inizialmente viene inserita una tupla all'interno del tuple space e successivamente viene effettuata una chiamata di tipo *tuple_space:rd* o *tuple_space:in*, i cui pattern sono compatibili con la tupla appena inserita. Viene dunque misurato il tempo trascorso tra tale chiamata e la ricezione della relativa risposta. Nel caso in cui si tratti di un'operazione di tipo *rd*, la tupla viene poi rimossa dal tuple space, nel caso di un'operazione di tipo *in* ciò non è necessario in quanto la tupla viene rimossa dall'operazione stessa.

Per quanto riguarda le prestazioni di recovery del sistema, il procedimento è leggermente più complesso: il test prevede di creare un tuple space, di inserirci all'interno una tupla particolare e di terminare il processo *owner* in modo consapevolmente anomalo. Per far ciò è stato inserito all'interno della funzione *loop* del processo *owner* la ricezione di un messaggio particolare che fa terminare il processo provocando un messaggio di tipo *{reply, crashing}*, il quale viene interpretato dal processo *supervisor* come una terminazione anomala del processo supervisionato e che dovrebbe scatenare un reistanzamento istantaneo di tale processo. A seguire, tramite una funzione di supporto, viene effettuata un'operazione *rd* ogni 10 microsecondi il cui pattern è compatibile con la tupla inserita prima della terminazione e appena la *rd* ottiene una risposta, viene calcolato il tempo trascorso tra l'invio del messaggio di crash e la ricezione di tale risposta.

All'interno del modulo *bench*, ognuno di questi test è ripetuto 1000 volte e vengono calcolati i tempi medi, minimi e massimi, per ciascuna operazione. Per rendere i risultati più stabili, i test contenuti in *bench* sono stati effettuati 10 volte, sono state calcolate le medie per le misure citate in precedenza e i risultati sono stati riportati in figura [18]

	AVERAGE	MIN	MAX
OUT	1.219 ms	1.114 ms	15.258ms
RD	3.062 μ s	0 μ s	103 μ s
IN	1.281 ms	1.024 ms	16.814 ms
RECOVERY	3.071 ms	0.819 ms	22.641 ms

Figura 18: risultati test sulle prestazioni

Dai risultati contenuti nella figura [18] si conferma il comportamento atteso del sistema. L'operazione *rd*, con una media poco superiore ai 3 microsecondi, risulta significativamente più veloce rispetto a *in* e *out*, le cui medie si aggirano attorno agli 1.2 millisecondi e 1.3 millisecondi rispettivamente. Tale differenza è giustificata dal fatto che *rd* effettua la sola lettura della tupla, senza apportare alcuna modifica allo stato del sistema, mentre *out* e *in* comportano rispettivamente inserimento e rimozione di una tupla dal tuple space causando il conseguente aggiornamento dello stato del sistema e il relativo salvataggio su disco di tale stato. Il tempo di recovery del sistema, invece, risulta sensibilmente più alto rispetto alle primitive riguardanti le tuple: esso misura il tempo trascorso tra l'interruzione e il ripristino del servizio, durante il quale viene ricaricato lo stato persistente del tuple space e ne viene verificata la responsività.

5 Conclusioni

Il lavoro svolto durante la realizzazione di questo progetto ha permesso di realizzare un'implementazione di un tuple space distribuito funzionante, coerente con il paradigma Linda e capace di supportare operazioni primitive come inserimento, lettura e rimozione di tuple. Il sistema è inoltre in grado di effettuare una recovery, seppur parziale, del proprio stato in caso di terminazioni anomale.

Allo stesso tempo, un'analisi delle prestazioni e dei comportamenti di questo sistema ne ha messo in evidenza alcune limitazioni strutturali. In particolare, ad esempio, in caso di richiesta di aggiunta di una nuova tupla all'interno del tuple space, la funzione di supporto *handle_out*, privilegerà le richieste *in* in attesa e, qualora una di esse venga soddisfatta, la tupla non verrà aggiunta all'interno dello spazio, compromettendo la possibilità che la stessa tupla possa soddisfare delle operazioni *out*, sempre in attesa. A ciò si aggiungono possibili criticità nella registrazione globale del tuple space che, qualora dovesse fallire, potrebbe portare alla creazione di un processo non raggiungibile dai nodi distribuiti. È inoltre contemplata la possibilità che possa presentarsi un caso di *race condition* tra una richiesta in lettura con timeout e l'invio contemporaneo della risposta a tale

richiesta. Un ultimo possibile punto critico è l'utilizzo di una lista come modalità di memorizzazione delle tuple presenti nello spazio: tale scelta semplifica molto l'implementazione ma può causare perdite di efficienza nella ricerca di pattern compatibili qualora il numero di elementi presenti nel tuple space sia elevato.

A partire da queste osservazioni, emergono diverse possibilità di miglioramento di questo sistema riguardanti la logica del suo comportamento, la sua robustezza e la sua efficienza. Un primo intervento potrebbe essere l'adozione dei comportamenti OTP standard come *gen_server* e *supervisor*, al posto delle implementazioni "manuali" utilizzate in questo progetto. Un secondo spunto potrebbe essere implementare la persistenza, in aggiunta a quella già presente dello stato dello spazio, anche della coda *Waiting* che consentirebbe, in caso di terminazione controllata o anomala, di recuperare anche le richieste in attesa al momento della terminazione. Dal punto di vista delle prestazioni del sistema potrebbero essere introdotti meccanismi di indicizzazione delle tuple presenti all'interno del tuple space in grado di ridurre i tempi di lettura quando il sistema contiene un elevato numero di elementi. Nell'ottica della maggiore tolleranza ai guasti, si potrebbe implementare la presenza di copie distribuite e aggiornate automaticamente dello stato del sistema su più nodi con l'eventuale possibilità di promuovere a *owner* uno di questi, in caso di terminazione inaspettata dell'*owner* precedente. Se invece si decide di mantenere la persistenza locale singola dello stato, si potrebbero implementare dei metodi di aggiornamento parziale, aggiornando solamente le componenti effettivamente modificate, portando a possibili benefici sia in termini di efficienza che di scalabilità.

In conclusione, pur presentando alcune limitazioni e diverse possibilità di miglioramento, l'implementazione realizzata risulta complessivamente funzionale, funzionante e coerente con gli obiettivi prefissati e dimostra l'efficacia di Erlang e del paradigma Linda per quanto riguarda gestione, comunicazione e coordinazione di processi concorrenti.